Gold Layer Patterns

> **Document Owner:** Data Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

The Gold layer contains business-ready dimensional models optimized for analytics. This document defines patterns for YAML-driven table creation, MERGE operations, and constraint management.

Golden Rules

ID	Rule	Severity	WAF Pillar
DA-01	Schema extracted from YAML (never generated)	Critical	Performance
DA-02	Surrogate keys via MD5 hash	Required	Performance
DA-03	Always deduplicate before MERGE	Critical	Performance
DA-04	SCD Type 2 for historical dimensions	Required	Performance

> **Cross-references:** PK/FK constraints → see DM-02, DM-03. LLM-friendly documentation → see DL-07, CM-02, CM-03.

Platform & Enterprise Prerequisites

> This document defines **solution patterns**. The following EA/PA rules are prerequisites.

Prerequisite	Rule IDs	Document
Unity Catalog managed tables	DL-01..05	12-unity-catalog-tables
Dimensional modeling in Gold	DM-01..03	06-data-modeling
LLM-friendly table/column docs	DL-07, CM-02, CM-03	12-unity-catalog-tables
Naming conventions	NC-01, NC-02	05-naming-conventions

Gold Layer Purpose

Characteristic	Description
Aggregated	Tailored for analytics
Business logic	Domain-aligned
Optimized	For query performance
Dimensional	Facts + dimensions model

YAML-Driven Schema

Extract column names from YAML. Never generate from memory.

```yaml
# gold_layer_design/yaml/billing/fact_usage.yaml
table_name: fact_usage
primary_key:
  columns: ['usage_id']
foreign_keys:
  - columns: ['workspace_id']
    references: 'catalog.gold.dim_workspace'
    ref_columns: ['workspace_id']
columns:
  - name: usage_id
    type: STRING
    nullable: false
    description: "MD5 hash of workspace_id + date + sku"
  - name: workspace_id
    type: STRING
    nullable: false
```

Extract Schema Pattern

```python
import yaml
from pathlib import Path

def get_gold_schema(domain: str, table_name: str) -> dict:
    yaml_file = Path(f"gold_layer_design/yaml/{domain}/{table_name}.yaml")
    with open(yaml_file) as f:
        return yaml.safe_load(f)

# ✅ CORRECT
gold_columns = [col['name'] for col in get_gold_schema("billing",
"fact_usage")['columns']]

# ❌ WRONG
gold_columns = ["usage_date", "workspace_id"]  # Might be incomplete
```

MERGE with Deduplication

Always deduplicate Silver source before MERGE.

```python
from delta.tables import DeltaTable

# Step 1: Deduplicate - keep latest record
silver_df = (
    spark.table(silver_table)
```

```
    .orderBy(col("processed_timestamp").desc())
    .dropDuplicates([business_key])
)

# Step 2: MERGE
delta_gold = DeltaTable.forName(spark, gold_table)

delta_gold.alias("target").merge(
    silver_df.alias("source"),
    f"target.{business_key} = source.{business_key}"
).whenMatchedUpdateAll(
).whenNotMatchedInsertAll(
).execute()
```

Critical Rule

Deduplication key MUST match MERGE key:

```
# ✅ CORRECT
silver_df.dropDuplicates(["store_number"])
merge_condition = "target.store_number = source.store_number"

# ❌ WRONG
silver_df.dropDuplicates(["store_id"])  # Different key!
merge_condition = "target.store_number = source.store_number"
```

PK/FK Constraints

> **Note:** PK/FK constraint standards are defined in DM-02 and DM-03. The pattern below shows the Gold layer implementation.

Apply constraints AFTER all tables exist.

```
def apply_constraints(spark, catalog: str, schema: str):
    # Step 1: Primary Keys
    spark.sql(f"""
        ALTER TABLE {catalog}.{schema}.dim_workspace
        ADD CONSTRAINT pk_dim_workspace
        PRIMARY KEY (workspace_id) NOT ENFORCED
    """)

    # Step 2: Foreign Keys (after PKs exist)
    spark.sql(f"""
        ALTER TABLE {catalog}.{schema}.fact_usage
        ADD CONSTRAINT fk_usage_workspace
        FOREIGN KEY (workspace_id)
        REFERENCES {catalog}.{schema}.dim_workspace(workspace_id)
```

```
        NOT ENFORCED
    """)
```

Surrogate Keys

```
from pyspark.sql.functions import md5, concat_ws

# Dimension: business key + effective timestamp
dim_df = dim_df.withColumn(
    "store_key",
    md5(concat_ws("||", col("store_number"), col("effective_from")))
)

# Fact: all grain columns
fact_df = fact_df.withColumn(
    "usage_id",
    md5(concat_ws("||", col("workspace_id"), col("usage_date"),
col("sku_id")))
)
```

SCD Type 2 Pattern

```
updates_df = (
    silver_df
    .orderBy(col("processed_timestamp").desc())
    .dropDuplicates([business_key])
    .withColumn("surrogate_key",
                md5(concat_ws("||", col(business_key),
col("processed_timestamp"))))
    .withColumn("effective_from", col("processed_timestamp"))
    .withColumn("effective_to", lit(None).cast("timestamp"))
    .withColumn("is_current", lit(True))
)

delta_gold.alias("target").merge(
    updates_df.alias("source"),
    f"target.{business_key} = source.{business_key} AND target.is_current
= true"
).whenMatchedUpdate(set={
    "record_updated_timestamp": "source.record_updated_timestamp"
}).whenNotMatchedInsertAll(
).execute()
```

LLM-Friendly Documentation

> **Note:** Documentation standards are defined in DL-07, CM-02, and CM-03. The patterns below show Gold layer examples.

Table Comment Pattern

```
[Description]. Business: [use cases]. Technical: [grain, source].
```

```
COMMENT 'Daily usage fact at workspace-SKU-day grain.
Business: Cost analytics and billing reports.
Technical: MD5 surrogate key, incremental from Silver.';
```

Column Comment Pattern

```
workspace_id STRING NOT NULL COMMENT
'FK to dim_workspace. Business: Cost allocation grouping.
Technical: Immutable after creation.';
```

Validation Checklist

Pre-Development

- ☐ Gold YAML exists
- ☐ All columns defined
- ☐ PK/FK specified

Pre-Merge

- ☐ Schema validation passed
- ☐ Deduplication key = MERGE key
- ☐ All columns mapped

Post-Deployment

- ☐ Table has COMMENT
- ☐ PK constraint applied
- ☐ FK constraints applied

Common Errors

Error	Cause	Solution
`Column does not exist`	Generated name	Extract from YAML

Error	Cause	Solution
`DELTA_MULTIPLE_SOURCE_ROW_MATCHING`	No dedup	Add `dropDuplicates()`
`Table does not have PK`	FK before PK	Apply PKs first

References

- Medallion Architecture
- Delta MERGE
- SCD Type 2